

EXECUTE_ENABLE

A Proposed Open Hardware Standard for Consequential Machine Interlock

Derek A. Hone Remnant Fieldworks Inc. — Westerville, Ohio derek@ownerremnantfieldworks.com executionproof.io

August 2026 — Working Specification v0.2.1 Status: Pre-publication draft. Independent review and formal standardization are future steps. Revision note: v0.2 corrects the signal polarity description (active-high with fail-safe low default, not “active-low”), reconciles ProofRecord emission responsibility with the PCE specification, adds actor_id and intent_lineage to the ProofRecord field list, corrects the Class II optoisolation requirement, defines HOLD behavior at the gate, and specifies the verifier-recognition cryptographic binding mechanism. v0.2.1 corrects all remaining absolute hardware bypass claims: the Abstract TPM comparison is tightened from “a hardware root that cannot be bypassed by software” to a scoped enforcement-point description; §1.3 replaces the absolute “no instruction set can change the physics” with the bounded formulation “designed so that software cannot authorize consequence through the protected path while the hardware gate is deasserted” and explicitly acknowledges known implementation attack surfaces (alternate physical paths, fault injection, debug interfaces); §9 removes the residual overclaim “hardware — which cannot be bypassed by software” and replaces it with “a hardware protected path designed to remain closed when authorization is absent.”

Abstract

We propose EXECUTE_ENABLE, an open hardware interlock standard defining a universal physical permission line for machines that produce consequential actions. A consequential action is a machine-initiated action that produces an effect on an external system, physical state, financial record, or persistent data store that is not automatically reversed by the action itself and would require human or machine intervention to undo. A compliant device may not energize an actuator, release a financial transaction, execute a deployment, or produce any defined consequential output unless an upstream verification component has asserted the EXECUTE_ENABLE signal.

The signal is **active-high with a fail-safe low default**: assertion is a positive HIGH driven by a compliant verifier; the safe and default state is LOW, maintained by a mandatory pull-down resistor. Any failure — power loss, disconnection, timeout, verification error, communication fault — allows the pull-down to return the signal to LOW, which prevents consequence. The burden is always on the verifier to assert; the system never has to prove it should be stopped.

EXECUTE_ENABLE is to physical consequence what the Trusted Platform Module (TPM) is to identity: a hardware enforcement point designed so that software cannot authorize consequence through the protected path while the gate is deasserted, one that persists across system states and answers a governing question the software layer alone cannot answer. The TPM answers *is this the machine it claims to be?* EXECUTE_ENABLE answers *does this machine currently hold physical permission to act?*

We describe the electrical specification, the logical protocol, the compliance requirements, the relationship to the ExecutionProof Hardware Root of Control (EPHRC) prototype, and what remains to be formally specified, independently implemented, and submitted for standardization.

1. The Problem

1.1 The Physical Gap

Every layer of modern computing governance is implemented in software. Access control is software. Authorization is software. Policy enforcement is software. Audit logging is software.

Software governance has a structural limitation: it can be bypassed by software with sufficient privilege. A compromised operating system can override a permission check. A malicious agent with root access can

disable a logging mechanism. A sufficiently motivated attacker — or a sufficiently confused autonomous agent — can find a path around a software gate.

This limitation is tolerable when the consequence of a bad action is reversible. It becomes intolerable when the consequence is physical: a robot arm moves and injures someone, an industrial valve opens and releases a hazardous substance, a payment executes and funds move, a drone actuates and leaves a controlled zone, a medical device dispenses and the dose cannot be recalled.

For physical and irreversible consequential actions, a software governance layer is necessary but not sufficient. The physical action requires a physical interlock.

1.2 What Exists Today

No universal hardware standard currently governs the permission to produce physical or irreversible consequence. What exists instead:

Emergency stop systems (industrial, robotic): physical kill switches that halt all operation. Binary — full operation or full halt. No graduated authorization, no verification, no cryptographic binding.

Safety relays and interlocks (manufacturing): hardware circuits that monitor physical conditions and halt operation when conditions fall outside tolerance. Responsive to physical state, not to authorization state.

Hardware Security Modules (finance, cryptography): generate, store, and use cryptographic keys with physical tamper resistance. Govern key operations, not consequential actions.

Trusted Platform Modules (computing): establish cryptographic identity and measure platform state. Answer *is this the expected system?* not *is this action authorized?*

Software kill switches (AI systems, cloud): API-level shutdown mechanisms. Implemented in software; bypassable by software; dependent on network connectivity.

None of these answer the question EXECUTE_ENABLE is designed to answer:

Does this machine currently hold physical permission to produce this consequence?

1.3 Why Hardware

The answer must be hardware for the same reason the TPM answer to identity is hardware: because the threat model includes software compromise.

A software-only verification layer can be disabled by a compromised privileged process, bypassed by a side-channel that directly addresses hardware, silenced by logging suppression, circumvented by replay of previously valid authorization tokens, or ignored by an autonomous agent that has gained sufficient privilege.

A correctly implemented EXECUTE_ENABLE interlock is designed so that software cannot authorize consequence through the protected path while the hardware gate is deasserted. The signal either is or is not asserted on the protected path. No privileged instruction, API call, or configuration parameter may cause consequence through that path while the gate is LOW.

This claim is bounded. Alternate physical paths, malicious board design, compromised firmware controlling a separate actuator path, fault injection, and accessible debug interfaces can undermine any hardware implementation that does not close those surfaces. The guarantee is not that hardware is universally unbypassable — it is that a correctly implemented interlock provides a protection that software alone cannot provide. The requirements in §3 and §5 (isolation, fail-safe resistor, response windows, Class IV redundancy) address known bypass vectors. Independent hardware validation is a required future step because these guarantees must be verified against a physical implementation, not assumed from a specification.

EXECUTE_ENABLE is designed to be the minimum necessary hardware primitive for physical consequence governance — simple enough to implement in any hardware platform, powerful enough to provide a guarantee that software alone cannot.

2. The Intellectual Lineage

2.1 The TPM as Category Precedent

The Trusted Platform Module (TPM) was defined by the Trusted Computing Group (TCG) in 2003. It established a hardware root of trust for platform identity — a tamper-resistant chip that stores cryptographic keys, measures platform state, and provides attestation that cannot be forged by software.

The TPM created a new category: hardware-anchored identity for computing systems. Before the TPM, identity was software-managed and therefore software-bypassable. After the TPM, a machine could make a claim about its identity that was physically grounded — not because the software said so, but because the hardware attested it.

TPM became ubiquitous. Most major general-purpose computing platforms now implement it. It is a NIST requirement for federal systems. It is embedded in the security architecture of Windows, Linux, macOS, mobile platforms, and cloud infrastructure.

The path from TCG specification to ubiquitous standard took approximately fifteen years. The governing question changed: from *does the software report this identity?* to *does the hardware attest it?*

2.2 USB and PCIe as Interoperability Precedents

USB (Universal Serial Bus, 1996) and PCIe (PCI Express, 2003) are interoperability standards. They define an electrical, mechanical, and protocol contract that any compliant device can satisfy. A USB device does not need to know what host it is connecting to. A PCIe device does not need to know what system it is installed in. Compliance with the standard is the contract.

These standards created categories. Before USB, peripheral connectivity was fragmented — serial ports, parallel ports, PS/2, proprietary connectors. USB collapsed that into one standard and made interoperability the default.

The EXECUTE_ENABLE standard follows this interoperability pattern. A compliant industrial robot does not need to know which verification system is upstream. A compliant payment terminal does not need to know which authority layer is asserting the signal. Compliance with the standard is the contract. The device responds to the signal; the verifier generates it; the protocol governs the exchange.

2.3 EXECUTE_ENABLE as the Consequence Root

TPM created a hardware root of trust for identity. EXECUTE_ENABLE proposes a hardware root of trust for consequence.

Standard	Question Answered
TPM	Is this the machine it claims to be?
EXECUTE_ENABLE	Does this machine currently hold physical permission to act?

These are different questions. TPM attestation does not tell you whether an authenticated machine is authorized to produce a specific consequence right now. EXECUTE_ENABLE does not tell you whether a machine is the machine it claims to be. Both are necessary; neither is sufficient alone.

Together, they form the hardware foundation for trusted autonomous action: the machine is verified (TPM), and the machine holds verified permission (EXECUTE_ENABLE).

3. The EXECUTE_ENABLE Signal

3.1 Governing Doctrine

Active-high. Fail-safe low default. Assertion requires positive verification.

EXECUTE_ENABLE is an active-high signal. Assertion — the state that permits consequence — is logic HIGH, driven by a compliant upstream verifier. The safe and default state is logic LOW, maintained by a mandatory pull-down resistor at the receiving device.

Any failure — power loss, disconnection, timeout, verification error, communication fault, hardware fault — allows the pull-down to return the signal to LOW, which is the safe state. The burden is always on the verifier to assert. The system never has to prove it should be stopped; it is stopped unless actively permitted.

This arrangement is distinct from the common convention of “active-low” signals (which activate at LOW voltage) and also distinct from active-high signals that default to HIGH unless pulled down. EXECUTE_ENABLE is active-high in function (HIGH = permitted) with a fail-safe low default in hardware (undriven = LOW = safe). Both properties must hold simultaneously. The electrical specification in §3.2 gives the precise voltage levels.

3.2 Electrical Specification

Signal name: EXECUTE_ENABLE **Logic family:** LVCMOS 3.3V (primary); 1.8V and 5V variants defined in §3.4 **Asserted state (consequence permitted):** Logic HIGH — 2.0V at receiving device input, measured at the device pin **Deasserted state (safe state):** Logic LOW — 0.8V at receiving device input, or high-impedance with pull-down holding the pin LOW **Pull-down resistor:** 10 k Ω to GND, required at the receiving device — ensures fail-safe low behavior on disconnection or floating signal **Maximum assertion duration:** As specified in the policy contract (§4.3); verifier must re-assert at the policy-defined heartbeat interval or the receiving device must enter the safe state **Signal rise time:** 10 ns (assertion — HIGH transition) **Signal fall time:** 5 ns (deassertion — LOW transition; safety transitions take priority, deassertion must complete faster than assertion) **Isolation:** Optoisolation recommended for Class II and required for Class III and above (see §5)

3.3 Fail-Safe Requirements

A compliant receiving device MUST satisfy the following fail-safe conditions:

- If EXECUTE_ENABLE is deasserted for any reason, the device MUST enter its safe state within the response window defined for its device class (§5).
- If the signal line is open (disconnected), the pull-down resistor MUST hold the input LOW and the device MUST treat this as deasserted.
- If the verifier communication channel fails, the device MUST NOT infer continued authorization. Silence is not permission. The device MUST enter the safe state.
- If the device undergoes a power cycle or reset, it MUST begin in the safe state and remain there until a valid assertion is received.
- The device MUST NOT implement any software override of a deasserted EXECUTE_ENABLE signal. No privileged instruction, API call, or configuration parameter may cause the device to produce consequence while the hardware signal is deasserted.

3.4 Multi-Voltage and Isolation Variants

1.8V variant: Asserted HIGH 1.2V; deasserted LOW 0.4V. Pull-down 10 k Ω to GND. Level-shifting to 3.3V logic required before connection to a 3.3V verifier output.

5V variant: Asserted HIGH 3.5V; deasserted LOW 1.0V. Pull-down 10 k Ω to GND. For legacy industrial systems.

Optoisolated variant: For high-power, high-voltage, or hazardous-energy applications. The LED side is driven by the verifier; the phototransistor side drives the receiving device input. Deasserted = LED off =

phototransistor off = device input pulled LOW by pull-down. Required for Class III and above.

4. The Logical Protocol

4.1 The Three-Party Model

EXECUTE_ENABLE defines three parties. No party may substitute for another.

The Verifier. The component that evaluates whether consequence is authorized and generates the EXECUTE_ENABLE assertion. The verifier implements the authorization logic — authority check, evidence check, constraint check, freshness check. It emits the ProofRecord. In an ExecutionProof deployment, the verifier is the ExecutionProof engine stack. The verifier must satisfy §4.4.

The Gate. The hardware component that receives the EXECUTE_ENABLE signal and enforces the physical interlock. The gate sits between the verifier output and the device. It passes consequence only when the signal is asserted by a recognized verifier. It logs its own assertion and deassertion events independently of the verifier's ProofRecord. The gate must satisfy §4.5.

The Device. The machine, actuator, system, or component that produces consequence. It receives the gate's output and responds correctly to the asserted or deasserted state. The device need not implement verification logic; it need only respect the signal and satisfy the fail-safe requirements in §3.3.

The verifier decides. The gate enforces. The device acts. None of the three may substitute for the others.

4.2 The Authorization Cycle

An EXECUTE_ENABLE authorization cycle consists of five steps:

Step 1 — Proposal. The device or autonomous agent proposes a consequential action and submits it to the verifier.

Step 2 — Evaluation. The verifier evaluates the proposal against authority, evidence, policy, constraints, and freshness. The evaluation produces one of three outcomes: ALLOW, HOLD, or DENY.

Step 3 — ProofRecord emission. Regardless of outcome, the verifier emits a tamper-evident ProofRecord binding the proposal, the evaluation result, the verifier identity, and the timestamp. The ProofRecord is the verifier's permanent artifact.

Step 4 — Signal assertion. If and only if the outcome is ALLOW, the verifier drives EXECUTE_ENABLE HIGH for the duration and scope specified in the policy contract. For HOLD or DENY, the signal is not asserted.

Step 5 — Consequence, suspension, or safe state. - ALLOW: the gate confirms the assertion is from a recognized verifier and passes it to the device; the device produces the authorized consequence. The gate logs the assertion event. - HOLD: the signal is not asserted. The device remains in its safe state. The gate logs the HOLD event. The authorization cycle is suspended and may resume after re-evaluation (see §4.2a). - DENY: the signal is not asserted. The device remains in its safe state. The gate logs the DENY event.

4.2a HOLD Resolution

A HOLD is not a permanent DENY. It is a suspension pending resolution — additional approvals, updated evidence, a change in policy or constraint state, or human review.

When a HOLD is issued, the gate MUST log the event with a timestamp and a reference to the verifier's ProofRecord. The gate MUST NOT assert EXECUTE_ENABLE for a HOLDed action until the verifier issues a new ALLOW decision following fresh independent evaluation. The gate MUST NOT treat HOLD as an automatic DENY that terminates the cycle.

The proposer may resubmit the action with updated inputs. The verifier performs a new, independent evaluation. If the new evaluation returns ALLOW, the authorization cycle proceeds from Step 4. If it returns DENY, the cycle terminates with DENY.

4.3 The Heartbeat Requirement

A single assertion is not a permanent authorization. EXECUTE_ENABLE is not a switch; it is a heartbeat.

For long-duration operations, the verifier must re-assert the signal at the policy-defined heartbeat interval. If the heartbeat lapses, the gate treats the signal as deasserted and the device enters the safe state.

The heartbeat interval is policy-specific. For high-speed industrial operations it may be measured in milliseconds; for lower-speed financial operations in seconds. The gate must enforce the heartbeat using an independent hardware timer — not software timing alone.

4.4 Verifier Requirements

A compliant verifier MUST:

1. Evaluate proposed actions against authority, evidence, constraints, and freshness before asserting EXECUTE_ENABLE.
2. Emit a tamper-evident ProofRecord for every evaluation, regardless of outcome. ProofRecords for DENY and HOLD are as important as ProofRecords for ALLOW.
3. Assert EXECUTE_ENABLE for at most the duration and scope specified in the policy contract. Indefinite assertion is not compliant.
4. Maintain the heartbeat at the policy-defined interval during continuous operations.
5. Deassert immediately on any evaluation outcome of DENY or HOLD.
6. Implement replay resistance: a proposal bearing a freshness nonce that has been previously consumed MUST NOT produce a new ALLOW decision. New evaluation requires a new proposal with a new nonce. This applies regardless of the prior outcome — ALLOW, HOLD, or DENY.
7. Sign ProofRecords using ML-DSA-65 (NIST FIPS 204) or a post-quantum successor algorithm.

4.5 Gate Requirements

A compliant gate MUST:

1. Assert EXECUTE_ENABLE to the device only when the signal is driven by a recognized compliant verifier. Verifier recognition is established through cryptographic binding: during provisioning, the gate stores the public key or certificate of its authorized verifier. The verifier signs each heartbeat assertion. The gate verifies the signature before treating any assertion as valid. An assertion without a valid signature from a provisioned verifier MUST be treated as absent. Verifier provisioning and key rotation are defined in §[future Provisioning Annex].
2. Block consequence from reaching the device when the signal is deasserted, for any reason including signal loss, verifier failure, power interruption, and hardware fault.
3. Monitor the heartbeat using an independent hardware timer. If the heartbeat lapses beyond the policy-defined interval, the gate MUST treat the signal as deasserted and enter the safe state.
4. For a HOLD outcome: log the HOLD event with timestamp and ProofRecord reference; withhold assertion; await re-evaluation; do not treat as permanent DENY.
5. Not implement any software override of a deasserted signal.
6. Fail closed on any ambiguous state. Ambiguity is not permission.
7. Log the gate's own assertion and deassertion events — gate identity, timestamp, outcome, and cryptographic hash of the associated ProofRecord — independently of the verifier's ProofRecord. Both records together form the complete evidentiary record of the authorization cycle.

5. Device Classifications

Different consequential device classes have different response time and isolation requirements. EXECUTE_ENABLE defines four classes.

Class I — Digital Consequence

Examples: Payment authorization release, software deployment gates, API execution gates, data release, access grant decisions.

Response window: 100 ms from deassertion to safe state. *Isolation:* Optoisolation optional. *Heartbeat:* 10 s interval.

Class II — Low-Force Physical

Examples: Light robotics, laboratory dispensing, consumer devices, low-power actuators.

Response window: 50 ms from deassertion to safe state. *Isolation:* Optoisolation recommended. *Heartbeat:* 1 s interval.

Class III — High-Force or High-Speed Physical

Examples: Industrial robotics, manufacturing equipment, drones, vehicles, high-power actuators.

Response window: 10 ms from deassertion to safe state. *Isolation:* Optoisolation required. *Heartbeat:* 100 ms interval. *Additional:* Mechanical safe state (brake, stop, return to home position) must be available without requiring a software command.

Class IV — Hazardous Energy or Irreversible

Examples: Medical devices, surgical robotics, chemical dispensing, power grid switching, munitions, orbital or launch systems.

Response window: 1 ms from deassertion to safe state. *Isolation:* Galvanic isolation required — optoisolation plus physical domain separation of verifier and consequence circuits. *Heartbeat:* 10 ms interval. *Additional:* Redundant independent gate required — 2-of-2 gate consensus before any assertion reaches the device. Formal verification of gate logic required before deployment. Independent hardware watchdog required.

6. The ProofRecord for EXECUTE_ENABLE

Every EXECUTE_ENABLE authorization cycle must produce a ProofRecord emitted by the verifier. The ProofRecord is the permanent tamper-evident artifact of the authorization decision. It must be generated before consequence is produced, not reconstructed afterward.

A compliant ProofRecord for an EXECUTE_ENABLE cycle contains:

- The proposed action (exact, bound to this authorization)
- The actor_id (cryptographic identity of the proposing agent)
- The verifier's evaluation result (ALLOW / HOLD / DENY)
- The policy contract under which the action was evaluated
- The authority chain that authorized the action
- The evidence that supported the decision
- The constraint envelope the action was evaluated against
- The freshness nonce (consumed by this evaluation)

- The expiration of this authorization
- The intent_lineage (when present; recommended — see PCE specification §5.3 for current status)
- The verifier identity and post-quantum signature over all prior fields
- The ProofRecord timestamp

The gate independently logs: the gate identity, the timestamp, the outcome (assertion released / suspension / assertion withheld), and a cryptographic hash binding the log entry to the verifier’s ProofRecord.

Together, the verifier ProofRecord and the gate log form the complete evidentiary record. A ProofRecord that omits any required field is not compliant. A ProofRecord generated after execution is not compliant. The proof comes before the power.

7. What Remnant Fieldworks Has Already Built

7.1 The EPHRC Prototype

The ExecutionProof Hardware Root of Control (EPHRC) is the first implementation of the EXECUTE_ENABLE gate concept. It is an FPGA-synthesized interlock with a SHA3-256-based authorization signal path and a fail-safe-low `execute_enable` output.

Current validated status: Behavioral simulation (7/7), synthesis (Yosys 0.33), place-and-route, and open-source bitstream generation for the XC7Z020 target. Formal verification of the fail-closed invariant by k-induction, 13 separately-named properties, unbounded. Adversarial RTL testing: 22 preregistered scenarios, 22/22 PASS. Formal mutation testing: 12 faults, 12/12 killed (10/12 primary, 12/12 strengthened after remediation; pre-remediation result preserved as a first-class research record).

Zenodo deposit: DOI 10.5281/zenodo.22003045.

Current gaps relative to this standard: The heartbeat mechanism, device-class response-window verification, the Class IV redundant gate requirement, the verifier-recognition cryptographic binding (Requirement 1 of §4.5), and the separate gate log artifact (Requirement 7 of §4.5). These are defined gaps between the current prototype and a standard-compliant gate.

Approved public claim: ExecutionProof now has a synthesizable SHA3-256-based FPGA enforcement prototype that has passed behavioral tests, synthesis, place-and-route, and bitstream generation for an XC7Z020 target. Physical board execution, vendor tool signoff, formal verification of the SHA3 implementation itself, and independent reproduction remain future validation steps.

7.2 The Verifier

The ExecutionProof engine stack (Authority $\rightarrow$ Evidence $\rightarrow$ Constraint $\rightarrow$ Control) with post-quantum ProofRecord signing via ML-DSA-65 on AWS KMS HSM is a software implementation of the verifier role. It is running and producing ProofRecords. Current gaps relative to §4.4: the heartbeat assertion mechanism is not yet a formally timed protocol component, the nonce registry is not yet a standalone persistent component, and the gate-independent log is not yet a separate artifact from the verifier ProofRecord.

7.3 Corpus

106 documented design-before-execution experiments across 12 research families and 16 public repositories: 95 PASS, 8 preserved FAIL, 3 special status, 4 validated FAIL $\rightarrow$ PASS remediations. 85 Zenodo depositions. Internal and founder-led; all claims are bounded by that provenance.

8. What Remains

Heartbeat hardware implementation. The gate requires an independent hardware timer. The EPHRC prototype does not yet implement this.

Verifier-recognition provisioning annex. The cryptographic binding specified in Gate Requirement 1 requires a full specification of key provisioning, storage, and rotation procedures for production deployment.

Device class response-window verification. Class III and IV response-window requirements (10ms, 1ms) require timing analysis and formal verification against the gate RTL.

Redundant gate protocol. The Class IV 2-of-2 requirement needs a defined consensus protocol for gate synchronization and disagreement resolution.

Physical board validation. EPHRC requires execution on a physical XC7Z020 board.

Vendor tool signoff. Vivado signoff — required for production deployment on Xilinx/AMD platforms — has not been conducted.

Independent reproduction. A third party has not yet reproduced the EPHRC build. A reproduction package is staged; the disclosure path (file-then-reproduce vs. reproduce-under-NDA) requires a legal determination.

Standard submission. Recommended primary bodies: IEC TC 65 (Industrial Process Measurement, Control and Automation) for the physical interlock standard; IEEE for the hardware specification; TCG as a secondary given the trusted-computing positioning. Submission requires this specification to first survive external technical review.

Reference gate implementation. A freely available, independently reproducible RTL reference gate is required for adoption. The EPHRC is a candidate; it requires documentation and packaging as a standalone reference before it can serve that role.

9. The Category This Creates

TPM created a hardware root of trust for identity. The world now asks, for every computing system: where is the TPM?

EXECUTE_ENABLE proposes to create a hardware root of trust for consequence. The world would ask, for every consequential machine: where is the EXECUTE_ENABLE gate?

For industrial equipment: does the robot hold a current physical permission to move? For financial systems: does the payment terminal hold a current physical permission to release funds? For medical devices: does the infusion pump hold a current physical permission to dispense? For autonomous vehicles: does the drive system hold a current physical permission to actuate? For drones: does the flight controller hold a current physical permission to produce thrust?

None of these questions currently have a standard hardware answer. EXECUTE_ENABLE is the proposal to create one. If adopted, the governing question shifts from:

“Does the software authorize this?”

to:

“Does the hardware hold current physical permission?”

That shift moves the enforcement point from software alone to a hardware protected path designed to remain closed when authorization is absent. That is a new category. Not a new feature. A new layer.

10. Patent Notice

This work is related to patent applications pending before the United States Patent and Trademark Office. Applications are pending — not granted. The portfolio contains 8 pending non-provisional utility applications and 48 provisional applications. Licensing inquiries: derek@ownerremnantfieldworks.com.

11. Honesty Statement

This document is a working standard proposal produced by the inventor and founder of Remnant Fieldworks Inc. It has not undergone peer review or standards body review. The prototype implementation described herein is internal and founder-led. Physical board validation, vendor signoff, and independent reproduction have not been completed. The claims regarding formal verification and adversarial testing results are accurate to the extent of the current work.

The TPM and USB/PCIe analogies are the author’s own characterizations to illuminate the category being proposed. They do not imply endorsement by or affiliation with the Trusted Computing Group, Intel, or any standards body.

We publish this specification not as a finished standard but as the opening proposal for a conversation that becomes more urgent with every autonomous system deployed into a domain where actions produce physical, financial, or irreversible consequence.

References

- Trusted Computing Group. (2003). *TPM Main Specification, Version 1.2*. TCG.
- USB Implementers Forum. (1996). *Universal Serial Bus Specification, Revision 1.0*. USB-IF.
- PCI-SIG. (2003). *PCI Express Base Specification, Revision 1.0*. PCI-SIG.
- IEC 61508. (2010). *Functional Safety of E/E/EP Safety-Related Systems*. International Electrotechnical Commission.
- NIST. (2024). *FIPS 204: Module-Lattice-Based Digital Signature Standard (ML-DSA)*. National Institute of Standards and Technology.
- NIST. (2015). *FIPS 202: SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions*. National Institute of Standards and Technology.
- Hone, D. A. (2026). *Proof Carrying Execution: A Protocol for Autonomous Action Authorization*. Remnant Fieldworks Inc. Working Specification v0.2.
- Remnant Fieldworks Inc. (2026). *EPHRC Hardware Root of Control — ExecutionProof FPGA Enforcement Prototype*. Zenodo. doi:10.5281/zenodo.22003045.
-

Remnant Fieldworks Inc. — Ohio C-Corp — Westerville, Ohio remnantfieldworks.com · executionproof.io
© 2026 Derek A. Hone / Remnant Fieldworks Inc. All rights reserved. Patent pending.